Part 1: Traditional Machine Learning (Scikit-Learn)

For Regression and SVM, the pipeline is always: **Import -> Split Data -> Initialize Model -> Fit -> Predict -> Evaluate.**

1. Linear Regression (Predicting a continuous number)

```
import numpy as np
import math
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.datasets import fetch_california_housing

# Load built-in California Housing dataset
data = fetch_california_housing()
X, y = data.data, data.target

# 1. Split Data
# Common arguments for train_test_split:
# - test_size: 0.2 (20%) or 0.3 (30%) are industry standards
# - shuffle: True (default) or False (important for time-series data)
# - stratify: y (ensures class balance in classification tasks)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Initialize the model
# Common arguments for LinearRegression:
# - fit_intercept: True (default) calculates the bias; set to False if data is centered
# - copy_X: True (default); False overwrites X to save memory on large datasets
model = LinearRegression()

# 3. Train the model
model.fit(X_train, y_train)

# 4. Make predictions
predictions = model.predict(X_test)

# 5. Evaluate
# Alternatives for regression evaluation:
# - mean_absolute_error (MAE): better for handling outliers
# - r2_score: shows the proportion of variance explained (0 to 1)
# - RMSE: sqrt(MSE) — same units as the target (used in LSTM exam, Section 5b)
mse = mean_squared_error(y_test, predictions)
rmse = math.sqrt(mse)
print(f"Mean Squared Error: {mse}")
print(f"Root Mean Squared Error: {rmse}")
```

```
Mean Squared Error: 0.5558915986952425
Root Mean Squared Error: 0.7455813830127751
```

2. Support Vector Machine (Classification)

```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.datasets import load_breast_cancer

# Load built-in Breast Cancer dataset (Classification)
data = load_breast_cancer()
X, y = data.data, data.target

# 1. Data setup
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Initialize the model with hyperparameters
# kernel: 'rbf' (default, non-linear), 'linear' (for high-dimensional data), 'poly' (polynomial), 'sigmoid'
# C: Penalty parameter. 1.0 (default). Lower values (0.1) = simpler models; Higher (100) = strict classification
# gamma: 'scale' (default), 'auto', or a float. Defines the influence radius of training points
model = SVC(kernel='rbf', C=1.0, gamma='scale')

# 3. Train the model
model.fit(X_train, y_train)

# 4. Predict categories
predictions = model.predict(X_test)

# 5. Evaluate
```

```
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy * 100}%")

# confusion_matrix: rows = true class, columns = predicted class
print("Confusion Matrix:\n", confusion_matrix(y_test, predictions))

# classification_report: precision, recall, f1-score per class
print(classification_report(y_test, predictions))
```

```
Accuracy: 94.73684210526315%
Confusion Matrix:
 [[37  6]
 [ 0 71]]
              precision    recall  f1-score   support

           0       1.00      0.86      0.93        43
           1       0.92      1.00      0.96        71

    accuracy                           0.95       114
   macro avg       0.96      0.93      0.94       114
weighted avg       0.95      0.95      0.95       114
```

Part 2: Deep Learning (Keras)

For Neural Networks, the pipeline is: **Define Architecture (Layers) -> Compile (Loss & Optimizer) -> Fit (Epochs & Batches).**

Section	Topic
Quick Reference	Loss table, `history`, `verbose`, imports, train/val/test
3	Dense / Feedforward
4	Basic CNN (MNIST)
4a	OpenCV image loading (exam preprocessing)
4b	CNN exam patterns (Section B No. 1)
5	Basic LSTM
5b	LSTM exam patterns (Section B No. 2)
6	CNN + LSTM (video)

3. Standard Neural Network (Dense / Feedforward)

Use Case: Tabular data (CSVs, spreadsheets). **Input Shape:** 1D vector `(features,)`.

Part 2 Quick Reference (read before Sections 3–6)

Imports: In TensorFlow 2, `from keras...` and `from tensorflow.keras...` are equivalent. Exam templates often use `keras`; this doc uses `tensorflow.keras`.

Label format	Example	Loss to use
Integer class IDs	`y = [0, 1, 2]`	`sparse_categorical_crossentropy`
One-hot encoded	`to_categorical(y)`	`categorical_crossentropy`
Binary 0/1	`y = [0, 1, 1, 0]`	`binary_crossentropy`
Continuous numbers	regression target	`mse` or `mean_squared_error`

`history` object (store with `history = model.fit(...)`):

- `history.history['accuracy']` / `['loss']` — training metrics
- `history.history['val_accuracy']` / `['val_loss']` — only when `validation_data=(X_val, y_val)` is passed

`verbose`: `verbose=1` (default, show progress), `verbose=0` (silent fit/predict)

`model.summary()` — call after building any model to print layer shapes and parameter counts.

`plot_model` — requires Graphviz installed (`pip install pydot graphviz`; on Windows also install Graphviz system binary).

Train / validation / test (CNN exam):

- **Validation split** = slice of training data via `train_test_split` (exam: 70 train / 30 val)
- **Test folder** = completely separate images never seen during training (exam: `Testing/` folder)
- These are three different sets — do not confuse them.

Time-series split: use `train_test_split(..., shuffle=False)` to keep chronological order (see Section 5b).

```
import numpy as np
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# --- model.evaluate() vs predict() + argmax ---
data = load_iris()
X, y = data.data, data.target
num_classes = 3
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
Y_test = to_categorical(y_test, num_classes=num_classes)

tiny_model = Sequential([
    Dense(32, activation='relu', input_shape=(X.shape[1],)),
    Dense(num_classes, activation='softmax')
])
tiny_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
tiny_model.fit(X_train, to_categorical(y_train, num_classes=num_classes), epochs=5, verbose=0)

# Option A: model.evaluate() — returns [loss, metric(s)] directly
loss, acc = tiny_model.evaluate(X_test, Y_test, verbose=0)
print(f"evaluate() -> loss: {loss:.4f}, accuracy: {acc:.4f}")

# Option B: predict() + argmax — needed when exam asks for per-class predictions or sklearn metrics
Y_pred = tiny_model.predict(X_test, verbose=0)
Y_pred_classes = np.argmax(Y_pred, axis=1)
Y_true_classes = np.argmax(Y_test, axis=1)
print(f"predict() + accuracy_score: {accuracy_score(Y_true_classes, Y_pred_classes):.4f}")
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[7], line 2
      1 import numpy as np
----> 2 from tensorflow.keras.utils import to_categorical
      3 from tensorflow.keras.models import Sequential
      4 from tensorflow.keras.layers import Dense

ModuleNotFoundError: No module named 'tensorflow'
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load built-in Iris dataset
data = load_iris()
X, y = data.data, data.target
num_features = X.shape[1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 1. Build the network architecture
model = Sequential()

# Input layer + First Hidden Layer
# activation: 'relu' (standard), 'tanh' (zero-centered), 'sigmoid' (often used in output)
model.add(Dense(128, activation='relu', input_shape=(num_features,)))

# Second Hidden layer
model.add(Dense(64, activation='relu'))

# Output layer
# activation: 'softmax' (multi-class), 'sigmoid' (binary), 'linear' (regression)
model.add(Dense(3, activation='softmax'))

model.summary()  # always check layer shapes after building

# 2. Compile the model
# optimizer: 'adam' (default), 'sgd' (classic gradient descent), 'rmsprop' (good for RNNs)
# loss: 'sparse_categorical_crossentropy' (integer labels), 'categorical_crossentropy' (one-hot), 'binary_crossentropy' (bi
model.compile(optimizer=Adam(learning_rate=0.001),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 3. Train the model — store history if you need to plot metrics later
# epochs: number of training iterations. Higher = potential overfitting
# batch_size: number of samples per gradient update. Common: 8, 16, 32, 64
history = model.fit(X_train, y_train, epochs=20, batch_size=8,
                    validation_data=(X_test, y_test), verbose=1)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 4s 73ms/step - accuracy: 0.5000 - loss: 0.9664 - val_accuracy: 0.7000 - val_loss: 0.7754
Epoch 2/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.6833 - loss: 0.6918 - val_accuracy: 0.8333 - val_loss: 0.5942
Epoch 3/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.9000 - loss: 0.5208 - val_accuracy: 0.8000 - val_loss: 0.4614
Epoch 4/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.9000 - loss: 0.4416 - val_accuracy: 0.9667 - val_loss: 0.3977
Epoch 5/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.9000 - loss: 0.3745 - val_accuracy: 0.8667 - val_loss: 0.3627
Epoch 6/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.9583 - loss: 0.3276 - val_accuracy: 0.8667 - val_loss: 0.3128
Epoch 7/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.9500 - loss: 0.2809 - val_accuracy: 0.8667 - val_loss: 0.2829
Epoch 8/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.9583 - loss: 0.2511 - val_accuracy: 0.9667 - val_loss: 0.2448
Epoch 9/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - accuracy: 0.9667 - loss: 0.2206 - val_accuracy: 0.9667 - val_loss: 0.2186
Epoch 10/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 30ms/step - accuracy: 0.9750 - loss: 0.1953 - val_accuracy: 0.9000 - val_loss: 0.2126
Epoch 11/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9417 - loss: 0.2196 - val_accuracy: 0.8667 - val_loss: 0.2637
Epoch 12/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.9417 - loss: 0.1768 - val_accuracy: 0.9667 - val_loss: 0.1746
Epoch 13/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9667 - loss: 0.1471 - val_accuracy: 0.9000 - val_loss: 0.1800
Epoch 14/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9667 - loss: 0.1504 - val_accuracy: 0.8667 - val_loss: 0.2272
Epoch 15/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 27ms/step - accuracy: 0.9667 - loss: 0.1349 - val_accuracy: 0.9667 - val_loss: 0.1448
Epoch 16/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9667 - loss: 0.1254 - val_accuracy: 0.9667 - val_loss: 0.1431
Epoch 17/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9833 - loss: 0.1135 - val_accuracy: 0.8667 - val_loss: 0.1501
Epoch 18/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9583 - loss: 0.1243 - val_accuracy: 0.9000 - val_loss: 0.1575
Epoch 19/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9583 - loss: 0.1180 - val_accuracy: 0.9000 - val_loss: 0.1504
Epoch 20/20
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9750 - loss: 0.1030 - val_accuracy: 0.9000 - val_loss: 0.1593
<keras.src.callbacks.history.History at 0x7a1b6579a000>
```

4. Convolutional Neural Network (CNN)

Use Case: Images or spatial grids. **Input Shape:** 3D tensor `(height, width, color_channels)`.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.datasets import mnist

# Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
X_test = X_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0

model = Sequential()

# 1. Feature Extraction
# filters: 16, 32, 64, or 128. More filters capture more complex patterns.
# kernel_size: (3,3) or (5,5) are most common. Smaller is usually better for detail.
# padding: 'valid' (default, no padding) or 'same' (keeps image size constant)
model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)))
# pool_size: (2,2) is standard to reduce data size while keeping important info
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# 2. Decision Making
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.summary()

# 3. Compile and Train
# metrics: 'accuracy' (standard), 'auc' (area under curve), 'precision'
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
history = model.fit(X_train[:2000], y_train[:2000], epochs=5, batch_size=32, verbose=1)
```

```
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
63/63 ━━━━━━━━━━━━━━━━━━━━ 3s 26ms/step - accuracy: 0.7085 - loss: 1.0208
Epoch 2/5
63/63 ━━━━━━━━━━━━━━━━━━━━ 2s 27ms/step - accuracy: 0.9190 - loss: 0.2704
Epoch 3/5
63/63 ━━━━━━━━━━━━━━━━━━━━ 3s 44ms/step - accuracy: 0.9540 - loss: 0.1684
Epoch 4/5
63/63 ━━━━━━━━━━━━━━━━━━━━ 2s 28ms/step - accuracy: 0.9640 - loss: 0.1257
Epoch 5/5
63/63 ━━━━━━━━━━━━━━━━━━━━ 3s 29ms/step - accuracy: 0.9780 - loss: 0.0855
<keras.src.callbacks.history.History at 0x7a1aec5a3890>
```

4a. Image Preprocessing with OpenCV (Section B No. 1)

Use Case: Loading custom images from class folders before feeding them into a CNN.

Pipeline: Loop folders → `cv2.imread` → RGB convert → resize → shuffle → stack arrays → reshape → normalize `/255` → `to_categorical`

Note: `MaxPooling2D` and `MaxPool2D` are the same layer (alias). Exam templates may import either.

```
import os
import random
import numpy as np
import cv2
import warnings
from keras.utils import to_categorical

warnings.filterwarnings('ignore')

# --- Load images from Training folder (one subfolder per class) ---
np.random.seed(1234)
directory = "path/to/Fruit/Training"          # exam: ../DATASET/No.1/Fruit/Training
classes = ["Watermelon", "Tomato", "Plum", "Mango Red", "Banana"]
num_classes = len(classes)
img_size = 100
all_arrays = []

for class_name in classes:
    path = os.path.join(directory, class_name)
    class_num = classes.index(class_name)
    for img_file in os.listdir(path):
        img_array = cv2.imread(os.path.join(path, img_file))
        img_array = cv2.cvtColor(img_array, cv2.COLOR_BGR2RGB)   # BGR (OpenCV default) -> RGB
        img_array = cv2.resize(img_array, (img_size, img_size))
        all_arrays.append([img_array, class_num])

# --- Load separate Testing folder (held-out test set — NOT the validation split) ---
directory2 = "path/to/Fruit/Testing"          # exam: ../DATASET/No.1/Fruit/Testing
all_arrays2 = []
for class_name in classes:
    path = os.path.join(directory2, class_name)
    class_num = classes.index(class_name)
    for img_file in os.listdir(path):
        img_array = cv2.imread(os.path.join(path, img_file))
        img_array = cv2.cvtColor(img_array, cv2.COLOR_BGR2RGB)
        img_array = cv2.resize(img_array, (img_size, img_size))
        all_arrays2.append([img_array, class_num])

# --- Build arrays, shuffle, normalize ---
random.shuffle(all_arrays)
X_train, Y_train = [], []
for features, label in all_arrays:
    X_train.append(features)
    Y_train.append(label)
X_train = np.array(X_train)

random.shuffle(all_arrays2)
X_test, Y_test = [], []
for features, label in all_arrays2:
    X_test.append(features)
    Y_test.append(label)
X_test = np.array(X_test)

# Reshape to (samples, height, width, channels) and scale pixels to 0~1
X_train = X_train.reshape(-1, img_size, img_size, 3) / 255
X_test = X_test.reshape(-1, img_size, img_size, 3) / 255
```

```
Y_train = to_categorical(Y_train, num_classes=num_classes)
Y_test = to_categorical(Y_test, num_classes=num_classes)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

4b. CNN — Exam Patterns (Section B No. 1)

Use Case: Custom image folders (see Section 4a), one-hot labels, validation split, training plots, architecture export.

Extra steps beyond Section 4: `to_categorical` → `train_test_split` (absolute counts) → `history = model.fit(..., validation_data=...)` → `plot_model` → plot accuracy/loss → test `accuracy_score`

Requires Graphviz for `plot_model` (see Part 2 Quick Reference).

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
from tensorflow.keras.utils import to_categorical, plot_model
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.optimizers import Adam

# Assumes X_train, Y_train, X_test, Y_test from Section 4a (or dummy data below)
img_size = 100
num_classes = 5
X = np.random.rand(100, img_size, img_size, 3).astype('float32')
Y = to_categorical(np.random.randint(0, num_classes, 100), num_classes=num_classes)

# --- a. Train / validation split (from training pool only — NOT the Testing folder) ---
# test_size=30 and train_size=70 are absolute sample counts (not percentages)
x_train, x_val, y_train, y_val = train_test_split(
    X, Y, test_size=30, train_size=70, random_state=1234
)

# --- b. Build, compile, train (store history for plotting) ---
model = Sequential()

model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(img_size, img_size, 3)))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))

model.compile(optimizer=Adam(learning_rate=0.001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=10,
    batch_size=16,
    verbose=1
)

# --- c. Export architecture diagram (.png) — needs Graphviz installed ---
plot_model(model, to_file='model_architecture.png', show_shapes=True, show_layer_names=True)
model.summary()

# --- d. Plot training & validation accuracy / loss from history ---
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
```

```
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()

# --- e. Evaluate on separate Testing folder (Section 4a X_test / Y_test) ---
X_test = np.random.rand(20, img_size, img_size, 3).astype('float32')
Y_test = to_categorical(np.random.randint(0, num_classes, 20), num_classes=num_classes)

Y_pred = model.predict(X_test, verbose=0)
Y_pred_classes = np.argmax(Y_pred, axis=1)
Y_true_classes = np.argmax(Y_test, axis=1)

print("Test Accuracy:", accuracy_score(Y_true_classes, Y_pred_classes))
print("Confusion Matrix:\n", confusion_matrix(Y_true_classes, Y_pred_classes))

# Alternative one-liner: loss, acc = model.evaluate(X_test, Y_test, verbose=0)
```

5. Long Short-Term Memory Network (RNN / LSTM)

Use Case: Time-series, text, sequential data. **Input Shape:** 2D tensor `(time_steps, features_per_step)`.

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Generating dummy time-series data
X_train = np.random.rand(100, 100, 1)
y_train = np.random.rand(100, 1)

model = Sequential()

# 1. Sequence Processing Layer
# units: complexity of internal memory (32, 64, 128 are common)
# return_sequences: False (default, for single output), True (to stack another LSTM on top)
# dropout: 0.2 (common to prevent overfitting in recurrent units)
model.add(LSTM(64, return_sequences=False, input_shape=(100, 1)))

# 2. Output layer
model.add(Dense(1, activation='linear'))

model.summary()

# 3. Compile
# loss: 'mse' (mean squared error), 'mae' (mean absolute error), 'huber' (robust to outliers)
model.compile(optimizer='adam', loss='mse')

# 4. Train
history = model.fit(X_train, y_train, epochs=5, batch_size=16, verbose=1)
```

```
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_di
  super().__init__(**kwargs)
7/7 ━━━━━━━━━━━━━━━━━━━━ 3s 40ms/step - loss: 0.2763
Epoch 2/5
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.1036
Epoch 3/5
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1005
Epoch 4/5
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0922
Epoch 5/5
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0911
<keras.src.callbacks.history.History at 0x7a1afa7b3ef0>
```

5b. LSTM — Exam Patterns (Section B No. 2)

Use Case: Real CSV time-series (e.g. COVID cases), scale data, build sequences, stacked LSTM + Dropout, inverse-transform predictions, RMSE, plot true vs predicted.

Pipeline: Select column → `MinMaxScaler` → `train_test_split(shuffle=False)` → `create_dataset()` → reshape to 3D → 3× LSTM + Dropout → fit → `inverse_transform` → RMSE → plot

`time` parameter: `time=1` uses 1 past step to predict the next. Increase to use more history (e.g. `time=3` uses 3 past months).

`mse` vs `mean_squared_error`: interchangeable in `model.compile()`.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import math
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

# --- a. Preprocessing ---
# df = pd.read_csv('../DATASET/No.2/COVID-19.csv')
# df = df[['India New cases']]

df = pd.DataFrame({'India New cases': np.cumsum(np.random.randint(100, 500, 30))})

plt.figure(figsize=(10, 5))
plt.plot(df['India New cases'])
plt.title('Trend Plot')
plt.xlabel('Month Index')
plt.ylabel('New Cases')
plt.show()

scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df)

# shuffle=False keeps time order intact (important for time-series)
train, test = train_test_split(scaled_data, test_size=0.2, random_state=0, shuffle=False)

def create_dataset(dataset, time=1):
    # time=1: use 1 previous step to predict next; increase time to use more past steps
    data_X, data_Y = [], []
    for i in range(len(dataset) - time - 1):
        a = dataset[i:(i + time), 0]
        data_X.append(a)
        data_Y.append(dataset[i + time, 0])
    return np.array(data_X), np.array(data_Y)

time_step = 1
train_X, train_Y = create_dataset(train, time_step)
test_X, test_Y = create_dataset(test, time_step)

train_X = np.reshape(train_X, (train_X.shape[0], train_X.shape[1], 1))
test_X = np.reshape(test_X, (test_X.shape[0], test_X.shape[1], 1))

# --- b. Build stacked LSTM (3 layers × 25 units, 20% dropout after each) ---
model = Sequential()

model.add(LSTM(25, return_sequences=True, input_shape=(time_step, 1)))
model.add(Dropout(0.2))

model.add(LSTM(25, return_sequences=True))
model.add(Dropout(0.2))

model.add(LSTM(25))
model.add(Dropout(0.2))

model.add(Dense(1, activation='linear'))

model.summary()

# --- c. Compile, train, evaluate ---
model.compile(optimizer='adam', loss='mse')   # same as loss='mean_squared_error'
model.fit(train_X, train_Y, epochs=100, batch_size=1, verbose=1)

train_predict = model.predict(train_X, verbose=0)
test_predict = model.predict(test_X, verbose=0)

train_predict = scaler.inverse_transform(train_predict)
train_Y = scaler.inverse_transform([train_Y])
test_predict = scaler.inverse_transform(test_predict)
test_Y = scaler.inverse_transform([test_Y])

train_rmse = math.sqrt(mean_squared_error(train_Y[0], train_predict[:, 0]))
test_rmse = math.sqrt(mean_squared_error(test_Y[0], test_predict[:, 0]))
print('Train RMSE:', train_rmse)
print('Test RMSE:', test_rmse)

plt.figure(figsize=(10, 5))
plt.plot(test_Y[0], label='True Data')
plt.plot(test_predict[:, 0], label='Predicted Data')
```

```
plt.title('True vs Predicted')
plt.xlabel('Time Step')
plt.ylabel('New Cases')
plt.legend()
plt.show()
```

6. CNN + LSTM (Spatiotemporal)

Use Case: Video (a sequence of images). **Input Shape:** 4D tensor `(frames, height, width, color_channels)`.

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import TimeDistributed, Conv2D, MaxPooling2D, Flatten, LSTM, Dense

# Generating dummy spatiotemporal (video-like) data
# Shape: (samples, time_steps, height, width, channels)
X_train = np.random.rand(10, 10, 64, 64, 3) # 10 samples, 10 frames each
y_train = np.random.randint(0, 5, 10)

model = Sequential()

# 1. Time-Distributed CNN (Process each frame individually)
# wrapper: TimeDistributed is necessary to apply the same CNN to every frame in the sequence
# input_shape: (time_steps, height, width, channels)
model.add(TimeDistributed(Conv2D(32, (3, 3), activation='relu'),
                          input_shape=(10, 64, 64, 3)))
# MaxPooling reduces spatial dimensions per frame
model.add(TimeDistributed(MaxPooling2D((2, 2))))
# Flatten converts the 2D frame maps into a 1D vector for the LSTM
model.add(TimeDistributed(Flatten()))

# 2. LSTM (Understand motion/patterns over the time dimension)
# units: 32, 64, or 128 (more units capture more complex temporal dependencies)
# return_sequences: False (standard for classification), True (if stacking another LSTM)
model.add(LSTM(64))

# 3. Output (Classification layer)
# activation: 'softmax' for multi-class classification (0-4 in this demo)
model.add(Dense(5, activation='softmax'))

model.summary()

# optimizer: 'adam' is robust for spatiotemporal tasks
# loss: 'sparse_categorical_crossentropy' since y_train contains integer class labels
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=5, verbose=1)
```

```
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/wrapper.py:27: UserWarning: Do not pass an `input_shape`/`inpu
  super().__init__(**kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 6s 6s/step - accuracy: 0.3000 - loss: 1.6261
Epoch 2/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.2000 - loss: 2.2549
Epoch 3/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - accuracy: 0.2000 - loss: 1.9447
Epoch 4/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - accuracy: 0.2000 - loss: 1.5443
Epoch 5/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - accuracy: 0.3000 - loss: 1.4630
<keras.src.callbacks.history.History at 0x7a1aed3ebf80>
```

Part 3: Reinforcement Learning — Q-Learning to Deep Q-Learning

Core idea (both versions): An agent learns by trial and error. At each step it picks an action, gets a reward, and updates its estimate of how good that action was in that situation.

The Q-value `Q(state, action)` = expected total future reward from taking `action` in `state`.

Step	What happens
1. Observe	Agent sees current `state`
2. Act	Choose action (epsilon-greedy: explore randomly or exploit best known action)
3. Reward	Environment returns `reward`, `next_state`, `done`
4. Update	Apply the **Bellman equation**: `target = reward + gamma * max(Q(next_state))`
5. Repeat	Until episode ends; loop over many episodes

	7a. Tabular Q-Learning	7b. Deep Q-Learning (DQN)
Memory	`Q` table — one value per (state, action) pair	Neural network — generalizes across similar states
Best for	Small discrete environments (grid world, few states)	Large / continuous states (pixels, CartPole, games)
Lookup Q-values	`Q[state, action]` or `np.argmax(Q[state])`	`model.predict(state)` → vector of Q-values per action
Update	Direct: `Q[s,a] += alpha * (target - Q[s,a])`	Indirect: `model.fit(state, target_vector, epochs=1)`
Exploration	Epsilon-greedy	Epsilon-greedy (identical logic)
Bellman target	Same formula	Same formula

Read 7a first to understand the loop, then **7b** shows the only change: replace the Q-table with a Keras network.

7a. Tabular Q-Learning (Traditional / Regular)

Use Case: Small environments where every state and action can be listed in a table.

Key hyperparameters:

- `alpha` (learning rate): how fast the table updates — 0.1 is common
- `gamma` (discount factor): how much future rewards matter — 0.95 or 0.99
- `epsilon` (exploration): probability of random action — 0.1 (10%) is common

Limitation: A Q-table cannot scale to image inputs or thousands of states — that is why we need Deep Q-Learning (Section 7b).

```
import numpy as np
import random

# --- Environment setup: 5 states, 2 actions (0=left, 1=right), goal at state 4 ---
n_states = 5
n_actions = 2
Q = np.zeros((n_states, n_actions))   # Q-TABLE: rows=states, columns=actions

alpha = 0.1    # learning rate — how quickly we overwrite old Q-values
gamma = 0.95   # discount factor — weight of future rewards
epsilon = 0.1  # exploration rate — 10% random actions

def env_step(state, action):
    if action == 0:
        next_state = max(0, state - 1)
    else:
        next_state = min(n_states - 1, state + 1)
    reward = 1.0 if next_state == n_states - 1 else 0.0
    done = next_state == n_states - 1
    return next_state, reward, done

# --- The Q-Learning loop (same structure reused in Deep Q-Learning) ---
for episode in range(200):
    state = 0
    done = False

    while not done:
        # A. Epsilon-greedy action selection
        if random.uniform(0, 1) < epsilon:
            action = random.randrange(n_actions)
        else:
            action = np.argmax(Q[state])          # best known action for this state

        # B. Take action in environment
        next_state, reward, done = env_step(state, action)

        # C. Bellman target
        target = reward
        if not done:
            target = reward + gamma * np.max(Q[next_state])

        # D. Update Q-table directly
        Q[state, action] = Q[state, action] + alpha * (target - Q[state, action])

        state = next_state

print("Learned Q-table:\n", np.round(Q, 2))
print("Best action per state:", np.argmax(Q, axis=1))
```

7b. Deep Q-Learning (DQN) — Neural Network replaces the Q-table

Use Case: Robotics, game playing, any environment with too many states for a table.

What changes from 7a:

7a (Tabular)	7b (Deep Q)
`Q = np.zeros((n_states, n_actions))`	`q_network = Sequential([Dense(...)])`
`np.argmax(Q[state])`	`np.argmax(q_network.predict(state)[0])`
`Q[s,a] += alpha * (target - Q[s,a])`	Build `target_f` vector, then `q_network.fit(state, target_f, epochs=1)`
`np.max(Q[next_state])`	`np.amax(q_network.predict(next_state)[0])`

What stays the same: epsilon-greedy exploration, Bellman target formula, episode loop, `gamma`, `reward`, `done`.

Note: There is no single `model.fit(X, y)` on a fixed dataset. The network trains one step at a time inside the game loop.

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import random

# --- Environment (same loop as Section 7a, but states are continuous vectors) ---
# In practice: env = gym.make("CartPole-v1")
state_size = 4
action_size = 2
gamma = 0.95
epsilon = 0.1

class DummyEnv:
    def reset(self): return np.random.rand(1, state_size)
    def step(self, action): return np.random.rand(1, state_size), 1, random.random() > 0.9, {}
env = DummyEnv()

# REPLACES Q-TABLE: neural network outputs one Q-value per action
q_network = Sequential()
q_network.add(Dense(24, activation='relu', input_shape=(state_size,)))
q_network.add(Dense(24, activation='relu'))
q_network.add(Dense(action_size, activation='linear'))   # linear = raw Q-scores, not probabilities
q_network.compile(loss='mse', optimizer=Adam(learning_rate=0.001))
q_network.summary()

# --- Same A→B→C→D loop as Section 7a ---
for episode in range(5):   # use 1000+ for real training
    state = env.reset()
    done = False

    while not done:
        # A. Epsilon-greedy (identical logic to 7a)
        if random.uniform(0, 1) < epsilon:
            action = random.randrange(action_size)
        else:
            q_values = q_network.predict(state, verbose=0)
            action = np.argmax(q_values[0])          # was: np.argmax(Q[state])

        # B. Take action
        next_state, reward, done, info = env.step(action)

        # C. Bellman target (identical formula to 7a)
        target = reward
        if not done:
            target = reward + gamma * np.amax(q_network.predict(next_state, verbose=0)[0])

        # D. Update — REPLACES Q[s,a] += alpha * (target - Q[s,a])
        target_f = q_network.predict(state, verbose=0)
        target_f[0][action] = target
        q_network.fit(state, target_f, epochs=1, verbose=0)

        state = next_state

print("Demo Loop Complete.")
```

```
Demo Loop Complete.
```